PLANT LEAF HEALTH ANALYSIS

A Beginner-Friendly Image Analysis Project

Submitted by: _____

Programme / Class: B.Sc. Biology (please verify your exact programme details)

Institution: _____

Academic Year: _____

Project report and supporting files for learning purposes

Certificate (Template)

This is a template for review by the institution. It should be signed only by an authorised supervisor after reviewing the work.

This is to certify that the project titled **“Plant Leaf Health Analysis”** has been prepared by _____ under the guidance of _____ during the academic year _____.

Supervisor signature: _____ Date: _____
Institution seal (if applicable): _____

Declaration (Template)

I declare that I have reviewed this project, understand its purpose and limitations, and will accurately describe which parts I completed myself and which tools or resources I used. I will not present unperformed experiments or unverified results as completed work.

Student signature: _____ Date: _____

Acknowledgement

I would like to thank my teachers and institution for encouraging project-based learning. I also acknowledge the developers and maintainers of the open-source Python libraries used in this educational prototype. Any final acknowledgement should reflect the support actually received.

Abstract

This project demonstrates a simple image-analysis application for plant-leaf photographs. A user uploads a JPG or PNG image, and the application calculates rough proportions of green-toned and brown-toned pixels across the entire image. The interface is built with Streamlit, while Pillow and NumPy handle image loading and pixel calculations. The prototype is intended to introduce basic image processing and software documentation. It is not a trained machine-learning classifier, does not identify a particular disease, and does not estimate disease probability. Its measurements are affected by lighting, shadows, background, camera settings, and leaf condition. A future version could use a labelled, appropriately licensed dataset and a validated classification model, evaluated on independent test data.

1. Introduction

Plants are important for food, ecosystems, and livelihoods. Visible changes in leaves can be associated with many factors, including disease, nutrient conditions, age, water stress, lighting, and physical damage. Image processing offers a way to measure visual features, but a simple colour measurement cannot determine the cause of a symptom. This project provides a small, transparent demonstration of how an image can be loaded and its pixel colours summarised using Python.

2. Problem Statement

Beginners may want to explore whether computer tools can help examine plant-leaf images. The goal here is to build an easy-to-run educational application that accepts a leaf photo and reports basic colour statistics. The project does not claim to solve agricultural diagnosis; it demonstrates a first step in image processing and documents what would be needed for a more reliable system.

3. Objectives

- Build a small image-upload interface.
- Read JPG and PNG images.
- Calculate approximate green-toned and brown-toned pixel proportions.
- Display measurements clearly to the user.
- Explain limitations and safe next steps for future research.

4. Background / Literature Review

Digital image processing commonly represents a colour image as a grid of pixels, with each pixel containing red, green, and blue (RGB) channel values. Simple threshold rules can group pixels by colour, but these rules are sensitive to illumination and image background. Modern plant-disease research often uses labelled image datasets and machine-learning or deep-learning models; such systems require careful dataset selection, separation of training and test data, evaluation, and independent validation. This prototype uses only threshold-based colour summaries and does not implement those more advanced methods.

5. Methodology

Step 1: The user selects a JPG or PNG leaf photo.
Step 2: Pillow opens the image and converts it to RGB format.
Step 3: NumPy converts pixel values into arrays.
Step 4: Simple rules mark pixels that appear green-toned or brown-toned.
Step 5: The app calculates the percentage of all image pixels matching each rule and displays the result.

The percentages refer to the whole image, including background—not to the leaf alone.

6. Tools and System Requirements

- Python 3.10 or newer
- Streamlit for the user interface
- Pillow for opening images
- NumPy for array-based calculations
- A computer capable of running Python and a modern web browser

Dependencies are listed in the included **requirements.txt** file.

7. Dataset Description

No training dataset is required for this prototype because it does not train a model. Users may upload their own leaf photographs for demonstration. If a genuine disease-classification model is developed later, use a clearly documented dataset with labels and permission for the intended use. Keep training, validation, and test images separated appropriately, and avoid reporting performance unless it has been measured on data not used for training.

8. Implementation

The main source file is **app.py**. It creates a Streamlit page, accepts an image upload, displays the image, converts it to an RGB array, and applies simple channel-threshold rules. The file **requirements.txt** lists the libraries. The **README.md** file gives installation and execution instructions. These files are included in the accompanying ZIP package.

9. Results and Discussion

The app is designed to show two numerical summaries: the percentage of pixels classified as green-toned and the percentage classified as brown-toned. No experimental result is stated in this report because performance depends on the actual images used and has not been established as a disease-detection measure. A brown-toned area may come from soil, a pot, shadows, or other objects; a green-toned area may come from the background. Therefore, the values must not be interpreted as disease probability or used to select a treatment.

10. Advantages

• Simple interface suitable for beginners.
• Easy-to-understand Python source code.
• Works with common JPG and PNG image formats.
• Demonstrates image upload, pixel arrays, and basic colour rules.
• Does not require model training or a large dataset.

11. Limitations and Responsible Use

This prototype does not recognise named plant diseases and has not been validated for agricultural use. Results vary with light, shadows, camera quality, background, leaf age, and image framing. The rules may count non-leaf pixels. Do not use these values to diagnose a crop or decide on pesticides. For suspected crop disease, seek advice from a qualified agricultural extension service or plant-health professional.

12. Future Scope

A future research version could isolate the leaf from the background, collect a labelled and appropriately licensed dataset, train a machine-learning model, and compare it with a simple baseline. It should use independent test data, document class balance and data sources, report appropriate metrics such as precision, recall, and confusion matrix, and clearly state the model's limits. Any real-world use would require expert review and validation across different crops, locations, and imaging conditions.

13. Conclusion

This project provides a small practical introduction to image processing with Python. It accepts a leaf image and calculates rough green-toned and brown-toned pixel proportions. The main learning outcome is understanding how images can be represented as arrays and processed with simple rules. It is not a plant-disease detector. More data, model development, careful evaluation, and expert validation would be needed before making disease-related claims.

14. References and Learning Resources

- Python documentation: https://docs.python.org/3/
- Streamlit documentation: https://docs.streamlit.io/
- NumPy documentation: https://numpy.org/doc/
- Pillow documentation: https://pillow.readthedocs.io/

Consult the current documentation and dataset licences before extending the project.

Appendix A: How to Run

1. Install Python 3.10 or newer.
2. Extract the project ZIP to a folder.
3. Open a terminal in that folder.
4. Run **pip install -r requirements.txt**.
5. Run **streamlit run app.py**.
6. Open the local browser address printed by Streamlit and upload a JPG or PNG image.

Appendix B: Submission Checklist

Before submitting, confirm your institution and programme details, review the report, and make sure your submission rules permit this project. Do not sign the certificate template on behalf of a teacher. Upload **app.py** as Code File, **requirements.txt** as Requirements File, this PDF as Project Report, and **README.md** as README File. The GitHub Repository field needs a real repository URL after you upload the project files to your own GitHub account. Do not enter a made-up link.